Speaker Verification ("Voice Lock") — Build Roadmap

Goal: The app should only transcribe the shop owner's voice. If a customer, a family member, or a TV/radio speaks, the app must ignore it — even while the owner is speaking, any other voice should not reach the billing pipeline.

Status today (2026-10-02): The app records audio → uploads to `/api/transcribe` → Sarvam transcribes whatever it hears. There is no speaker check. Tap-to-talk and Android's `voice_recognition` mic source reduce noise, but they do NOT identify the speaker. This roadmap adds that layer.

Who this is for: The teammate who will build it. Every section says what to build, where it plugs into the existing code (exact file paths), and how to know it's done. Read top to bottom; phases are in order.

---

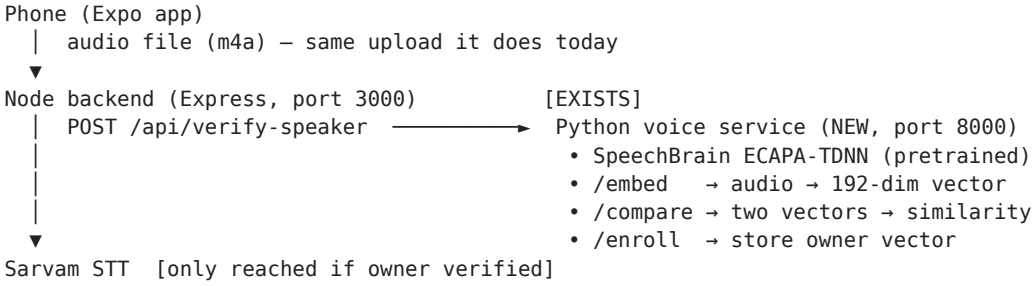
1. What we are building (one paragraph)

A small speaker verification service: the owner records ~15 seconds of their voice once (enrollment). The service converts that audio into a "voice fingerprint" (a vector of numbers). Later, every recording the app captures is also converted to a vector and compared with the owner's vector. Similar enough → it's the owner, send the audio to Sarvam. Not similar → drop it, and (optionally) buzz the phone so the owner knows it wasn't heard.

This is a classification-threshold problem, not a "recognise words" problem. We don't need to train a speech model — pretrained speaker-embedding models already exist. Our job is: enrollment flow, embedding storage, a comparison endpoint, and gating the existing pipeline. Do NOT plan to train a model from scratch.

---

2. Architecture (recommended)



Why server-side (Python) and not on-device:

- SpeechBrain (or HuggingFace `speechbrain/spkrec-ecapa-voxceleb`) gives a pretrained ECAPA-TDNN model that is SOTA-ish for this task and runs on CPU. On-device would mean TensorFlow Lite conversion, model quantisation, and fighting RN module integration — weeks of extra work for a first version.
- Our flow is already network-first (Sarvam is a cloud API). If the network is down, nothing bills anyway — so no new offline requirement is created.
- Embedding one clip takes ~150–300 ms of CPU. Fine next to the 1–2 s Sarvam call.
- Later (optional Phase 6) the same model can be exported to TFLite for on-device.

Decision made: Python FastAPI microservice, called by the Node backend over HTTP, exactly like Sarvam is called today. One more service on the same laptop.

---

3. Repo layout to create

```
mvp-project-sarvam/
├── backend/                    [EXISTS — Node/Express]
├── mobile-app/                 [EXISTS — Expo]
└── voice-service/              [NEW — you create this]
    ├── requirements.txt
    ├── main.py                 # FastAPI app
    ├── speaker.py              # model loading, embed, compare logic
    ├── store.py                # where owner embeddings are saved (JSON or SQLite)
    └── tests/
        └── test_speaker.py
```

`requirements.txt` (pin versions when you install):

```
fastapi
uvicorn[standard]
speechbrain
torch            # CPU wheel is fine; ~200 MB
torchaudio
python-multipart # file uploads
numpy
scipy            # to read m4a via soundfile/ffmpeg
```

Audio format note: the app uploads m4a (AAC). Python needs it decoded to PCM. Simplest reliable path: ffmpeg on the host (backend laptop already has ffmpeg if you install it; add `soundfile` + `ffmpeg-python`, or shell out to `ffmpeg -i in.m4a -ar 16000 -ac 1 out.wav`). 16 kHz mono is what ECAPA expects. Check ffmpeg is installed on Windows and available on PATH in the service's environment — this is the #1 thing that will break your first run.

---

4. The model (what to use, exactly)

Primary: `speechbrain/spkrec-ecapa-voxceleb` (ECAPA-TDNN, trained on VoxCeleb — ~7000 speakers). Load with SpeechBrain's `EncoderClassifier`:

```
from speechbrain.inference.speaker import EncoderClassifier
model = EncoderClassifier.from_hparams(
    source="speechbrain/spkrec-ecapa-voxceleb",
    savedir="pretrained_models/ecapa",
    run_opts={"device": "cpu"},
)
embeddings = model.encode_batch(waveform)   # waveform: torch tensor [1, T], 16 kHz
```

Returns a 1×192 vector per utterance. Normalise (L2) before comparing.

Comparison: cosine similarity, threshold around 0.25–0.35 for VoxCeleb- pretrained ECAPA. Start at 0.30, tune in Phase 5. Do not ship a threshold you haven't tested on real recordings from the shop.

Why not train our own model: we have one positive speaker and no negatives dataset. Pretrained embeddings + threshold is the standard few-shot approach. Custom training (fine-tuning) is a later experiment, not part of this roadmap.

---

5. API contracts (build the service to these exact shapes)

The Node backend will call these, so the shapes are the contract.

POST /enroll

multipart form, field `audio` (m4a/wav), field `speaker_id` (use `"shop_1_owner"`). Response:

```
{ "speaker_id": "shop_1_owner", "embedding_stored": true, "samples": 3, "similarity_mean": 0.82 }
```

Behaviour: call this 3–5 times with different clips of the owner; store the mean embedding (re-normalised). Storing the mean is more stable than any single sample. Keep the individual samples too for later tuning.

POST /verify

multipart form, field `audio`, field `speaker_id`. Response:

```
{ "speaker_id": "shop_1_owner", "score": 0.41, "is_speaker": true, "threshold": 0.30 }
```

`is_speaker = score >= threshold`. Never throw on "not the speaker" — that's a valid, expected result. Only throw on real errors (bad audio, model missing).

GET /health

Response `{"status": "ok", "model": "ecapa-voxceleb"}` — for the Node backend's boot check (like `checkWhatsAppConfig`).

Timeout discipline: embed+compare must respond in < 2 s on laptop CPU. If you're over that, you're re-loading the model per request — load it once at startup and keep it in memory.

---

6. Node backend changes (where it plugs in — exact files)

All in `backend/src/`:

- `services/voiceVerify.js` **(NEW)** — a small client like `transcribe.js`: reads `VOICE_SERVICE_URL` from `.env` (default `http://127.0.0.1:8000`), posts the audio file with `form-data`, returns `{ is_speaker, score }`. Also export `checkVoiceService()` for boot logging (see pattern in `services/whatsapp.js:82`).
- `routes/sale.js` — `POST /api/enroll-voice` **(NEW route):** accepts the same multipart upload as `/api/transcribe` (multer is already set up there, see `routes/sale.js:42`), forwards to the Python `/enroll`, returns its JSON.
- `routes/sale.js` — `POST /api/verify-and-transcribe` **(NEW route):** the same body/behaviour as `/api/transcribe`, but first calls the Python `/verify`. If `is_speaker` is false → return `{"text": "", "speaker_verified": false}` with HTTP 200 (NOT an error — the app treats "not heard" like silence). If true → run the existing `transcribeAudio()` and add `"speaker_verified": true` to its response.

Why a new route instead of editing `/api/transcribe`: the four-step typed flow (`RecordSaleScreen`) uses the old route and doesn't want gating. Additive routes, same pattern the hands-free `/utterance` route followed.

- `routes/sale.js` — **gate** `/api/utterance` **too (same file, later):** the hands-free screen's items/number path. `utterance` receives text, not audio — the gate belongs upstream in the app (Phase 4), because by the time text reaches `/utterance` the audio is gone. Do not try to gate it here.
- `.env` **(document, don't commit):** `VOICE_SERVICE_URL=http://127.0.0.1:8000` and `VOICE_ENROLLED=true/false` so the app can know whether to offer enrollment. Actually return enrolled-state from an API instead (see below).
- `routes/sale.js` — `GET /api/voice-status` **(NEW, tiny):** asks the Python service whether `shop_1_owner` has an embedding stored; returns `{"enrolled": true}`. The app uses this on the HandsFree screen to decide between "record enrollment" vs "normal use".

---

7. Mobile app changes (exact files)

- `services/api.js` — add `verifyAndTranscribeAudio(fileUri)` (clone of `transcribeAudio` at `services/api.js:14`, new URL) and `enrollVoice(fileUri)` and `getVoiceStatus()`.
- `services/useTapToTalk.js` **and** `useContinuousListening.js` — change the `onSegment` contract? No: keep `onSegment(uri)` as-is. The gate goes in `screens/HandsFreeScreen.js` `onSegment` (around line 56):

replace the call to `transcribeAudio(uri)` with `verifyAndTranscribeAudio(uri)`. If `speaker_verified === false`: set a brief UI state "Awaaz pehchani nahi gayi" (small toast text, ~2 s) and return — do NOT set `lastHeard`, do NOT send to `sendUtterance`. Silently dropping is also acceptable for v1; the toast is a nice-to-have so the owner knows to tap again or speak up.

- `screens/HandsFreeScreen.js` — on mount, call `getVoiceStatus()`. If not enrolled, render an enrollment panel above the mic button: "Apni awaaz register karein — 3 baar 5-second bolna hoga" with a record button that calls `enrollVoice()` three times (say a random sentence each time: show the sentence on screen, e.g. "Aaj mausam bada suhana hai", "Do kilo aam aur ek sabun", "Roz ka hisaab likha jata hai"). After the third call, re-check status and hide the panel. Keep it on the tap-mode recorder (5 s fixed duration is fine — `record({ forDuration: 5 })`).

UX rule (important): verification failure must NEVER block the countdown→send flow mid-sale. If items are already on the bill and the next utterance fails verification, just don't add anything. Don't reset, don't error-out.

---

8. Phases, in order, with "done" criteria

Phase 0 — Setup (half a day)

[ ] `voice-service/` repo created, requirements installed, ffmpeg works (`ffmpeg -version` in the service's terminal).

[ ] Model downloads once and loads in < 60 s at startup.

Done when: `uvicorn main:app --port 8000` serves `/health`.

Phase 1 — Core service (1–2 days)

[ ] `/embed`, `/verify`, `/enroll` implemented per the contracts above.

[ ] Model loaded once at startup (measure: two verifies back-to-back, both < 2 s).

[ ] `tests/test_speaker.py`: enroll with 3 clips of your own voice, verify with a 4th clip of yourself (expect `is_speaker: true`), verify with a YouTube clip / different person (expect false).

Done when: tests pass from the command line.

Phase 2 — Node integration (1 day)

[ ] `services/voiceVerify.js`, the three new routes, boot-time `checkVoiceService()` log line in `server.js`.

[ ] voice-service also auto-started: add a `start-all.cmd` at repo root that runs backend + voice-service + ngrok together (the project runs three processes today; make it four, or use `concurrently`).

Done when: `curl -F audio=@me.wav http://127.0.0.1:3000/api/verify-and-transcribe` returns real Sarvam text for the owner and `{"text":""}` for a stranger clip.

Phase 3 — App enrollment UI (1–2 days)

[ ] Enrollment panel on HandsFreeScreen (specs in §7.3).

[ ] Status check on mount.

Done when: a fresh install of the app walks the owner through enrollment and then works normally, all on the phone over Wi-Fi.

Phase 4 — Gate the live flow (1 day)

[ ] `onSegment` uses `verifyAndTranscribeAudio`; rejected audio never reaches `sendUtterance`; rejected audio also never shows in `lastHeard`.

[ ] Test in tap mode first; then flip auto mode on and test with music playing in the background.

Done when: music/TV transcript stays empty; owner's speech over the same music still works.

Phase 5 — Tuning & robustness (2–3 days, needs real data)

[ ] Collect ~20 real clips at the shop: 10 owner (varied: morning/evening, different distances), 10 others (customer/family/radio).

[ ] Sweep threshold 0.20→0.45 in 0.05 steps; print a small table of false-accept / false-reject counts. Pick the threshold where false-accept = 0 (a wrong person's speech must never be billed) while false-reject is still < ~15% (annoying but safe direction).

[ ] Log every verify result (timestamp + score, no audio) to a JSONL file for a week of real use before trusting the numbers.

Done when: threshold documented in `voice-service/README.md` with the evidence table.

Phase 6 (optional, later) — On-device

[ ] Export the same ECAPA model to TFLite (quantise to int8), wrap in a small Expo module (`expo-dev-client` build already exists in this repo), do embed+compare on the phone. Only worth it if offline/no-server becomes a requirement. Skip for MVP.

---

9. Failure & UX rules (decide these with the team, in writing)

- **False accept (stranger billed):** catastrophic → threshold must bias here.
- **False reject (owner ignored):** annoying → app shows the "not recognised" hint; owner taps and repeats. Never auto-retry in a loop.
- **Voice service down:** FAIL OPEN or FAIL CLOSED? Recommendation: fail open with a warning (behave like today, log loudly) — a dead verification service shouldn't stop all billing. Mark it in the response so the app can show a small "voice-lock off" badge.
- **Re-enrollment:** allow from a Settings screen later; for now, delete the stored embedding file and re-run enrollment.

---

10. Risks / gotchas the teammate will hit

- **m4a decoding in Python** — solve with ffmpeg subprocess; test the exact byte stream the app uploads (Expo writes `.m4a`, see `services/api.js:19`).
- **Very short clips:** ECAPA embeddings on <1 s of audio are noisy. Enforce a minimum of ~1.5 s of actual audio in `/verify` (check after decode); shorter → return `is_speaker: false` with a `"reason": "too_short"`.
- **The mic pipeline changed:** recordings use Android `voice_recognition` source (noise-suppressed). Enroll with the SAME pipeline the app will use — enroll from the phone, never from a laptop mic file. Otherwise the embeddings sit in slightly different acoustic conditions.
- **Cold start:** first `/verify` after boot can take 2–4 s (lazy model init). Warm it up: call `/verify` with 0.5 s of silence at service startup.
- **Two services on one laptop:** Node is on 3000, Python on 8000 — and the phone must reach BOTH through the same Wi-Fi IP as `BASE_URL` (`http://<laptop-ip>:8000`). Windows Firewall will prompt for Python — allow on Private AND Public (the hotspot registers as Public; this bit us already with Node).

---

11. What "done" looks like end-to-end (demo script)

1 Fresh app → enrollment panel → owner records 3 sentences.

2 Music playing. Customer speaks "5 kilo cheeni teen sau rupaye" → nothing appears on the bill.

3 Owner speaks the same sentence → items appear.

4 Owner dictates the customer's number → countdown → bill generated.

5 Voice-service log shows verify scores for every clip.

Read order for the teammate: §1 → §2 → §4 → §5 → your Phase (§8) → §10. Everything else is reference. Ask the project owner for: laptop access, the Twilio/Sarvam `.env` (already configured), and a week of shop audio for Phase 5.